Virtual Vahana

ADAS Simulation with Automatic Braking using MATLAB

Team Lead:

Manjare G

Team Members:

Ayesha Jumana S

Dhaareni K

Harshada S R

Naveena Mary S

Elakkiya K

Joshika A S

Jerlin Shiny A

Jessica M

Dhanyavarushine S

Department of Electrical and Electronics Engineering

Kumaraguru College of Technology

TABLE OF CONTENTS

1. Problem Statement

2. System Overview

3. Methodology

4. Algorithms & Calculations

5. System Architecture

6. Results & Observations

7. Challenges

8. Future Scope

9. Conclusion

1. Problem Statement

Road traffic accidents are consistently ranked among the leading causes of global fatalities and serious injuries. A primary contributing factor to these collisions is delayed driver reaction times to sudden obstacles, pedestrians, or leading vehicles. In highly dynamic driving environments, human drivers frequently fail to respond with adequate speed due to various human factors, including distraction, fatigue, cognitive overload, or sudden medical emergencies.

This project aims to address this critical safety gap by designing and simulating an Advanced Driver Assistance System (ADAS). The proposed system is engineered to:

- Continuously detect stationary or moving obstacles in the vehicle's forward trajectory.
- Mathematically compute the Time to Collision (TTC) in real-time.
- Provide preemptive, timely warnings to alert the driver of impending danger.
- Autonomously engage the vehicle's braking system in critical situations where driver intervention is absent or insufficient.

The ultimate objective is to enhance overall road safety by actively reducing the dependency on human reaction times during high-stress, emergency scenarios.

2. System Overview

The proposed system utilizes MATLAB to create a robust, kinematic simulation of a vehicle approaching a stationary obstacle. Rather than relying on physical hardware, this project mathematically models the physics of driving to validate ADAS logic.

Key Functionalities include:

- Continuous Distance Monitoring: Tracking the diminishing gap between the ego-vehicle and the obstacle at every time step of the simulation.
- Real-time TTC Calculation: Dynamically updating the time remaining before impact based on current velocity and distance.
- Threshold-based Decision Making: Utilizing predefined logical thresholds to dictate the vehicle's operational state.
- Multi-stage Interventions: Generating visual/audible dashboard warnings before escalating to autonomous mechanical intervention (braking).

This system successfully mimics real-world Automatic Emergency Braking (AEB) and Forward Collision Warning (FCW) features in a simplified, highly visual computational environment.

3. Methodology

The simulation is built upon a continuous computational loop that updates vehicle parameters at discrete time steps.

1. Initialization: The environment is established by defining the starting coordinates of the vehicle and the obstacle. The vehicle's initial velocity is set to a standard cruising speed.

2. Distance Calculation: At each time step, the system calculates the exact spatial gap: $Distance = Position_{obstacle} - Position_{vehicle}$.

3. TTC Calculation: The system divides the current distance by the current speed to find the Time to Collision.

4. Decision Logic: The core algorithm evaluates the TTC against safety thresholds:

    - TTC > 10 seconds: The vehicle is in a "Safe" state. Speed is maintained.

    - 5 < TTC $\leq$ 10 seconds: The system enters a "Warning" state. The driver is alerted, but no autonomous braking occurs.

    - TTC $\leq$ 5 seconds: The system identifies a "Critical" state and applies a deceleration factor to simulate automatic braking, bringing the speed to zero before impact.

5. Visualization: MATLAB's plotting functions are updated dynamically to show the vehicle's trajectory, the shrinking distance, and the changing TTC in real-time.

4. Algorithms & Calculations

Kinematic Equations:

- Time to Collision (TTC): $TTC = Distance / Speed$
  Braking Kinematics (when TTC ≤ 5):

  $Speed_{new} = Speed_{old} - (Deceleration * dt)$
  $Position_{new} = Position_{old} + (Speed * \Delta t)$

Decision Algorithm (Pseudocode):

WHILE Position < Obstacle Position AND Speed > 0:

1. Measure Distance

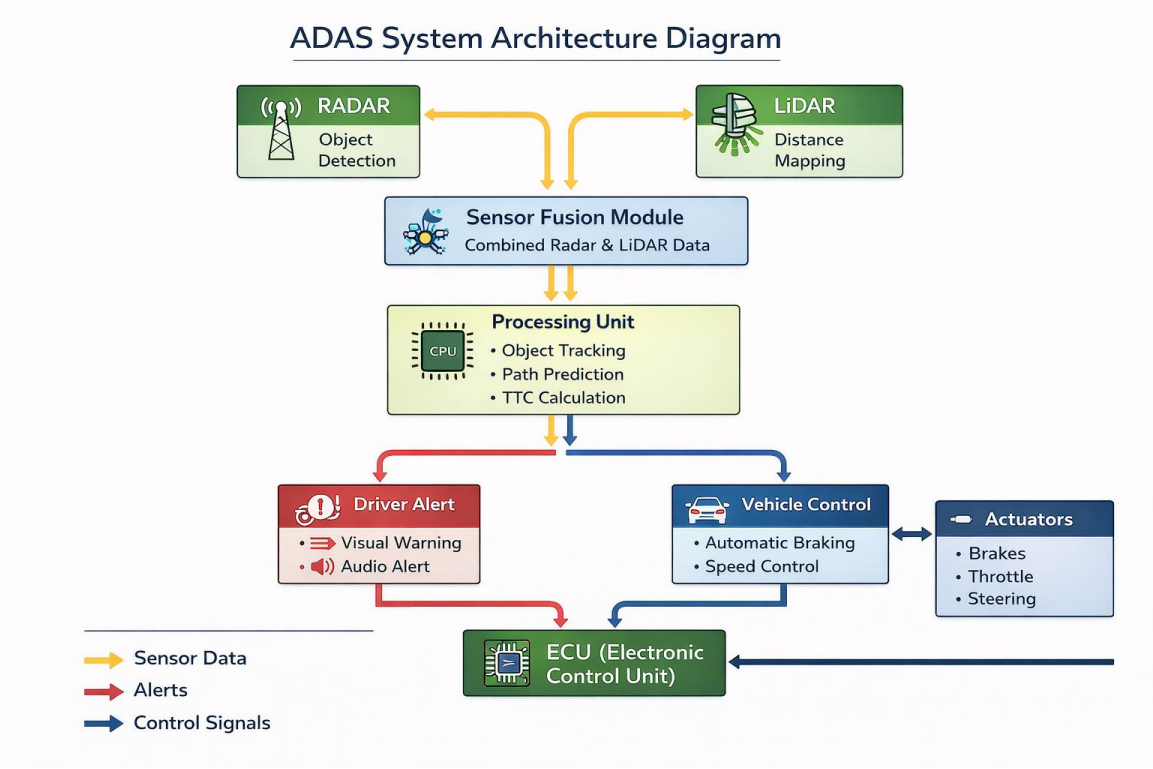
2. Calculate TTC

3. IF TTC > 10 THEN State = Normal
4. ELSE IF TTC > 5 AND TTC $\leq$ 10 THEN State = Warning Display
5. ELSE IF TTC $\leq$ 5 THEN Apply Deceleration (Brake)
6. Update Vehicle Position & Speed
7. Plot Data
8. END WHILE

5. System Architecture Breakdown

While operating in a simulated MATLAB environment, the system is architected to mirror real-world automotive software modules:

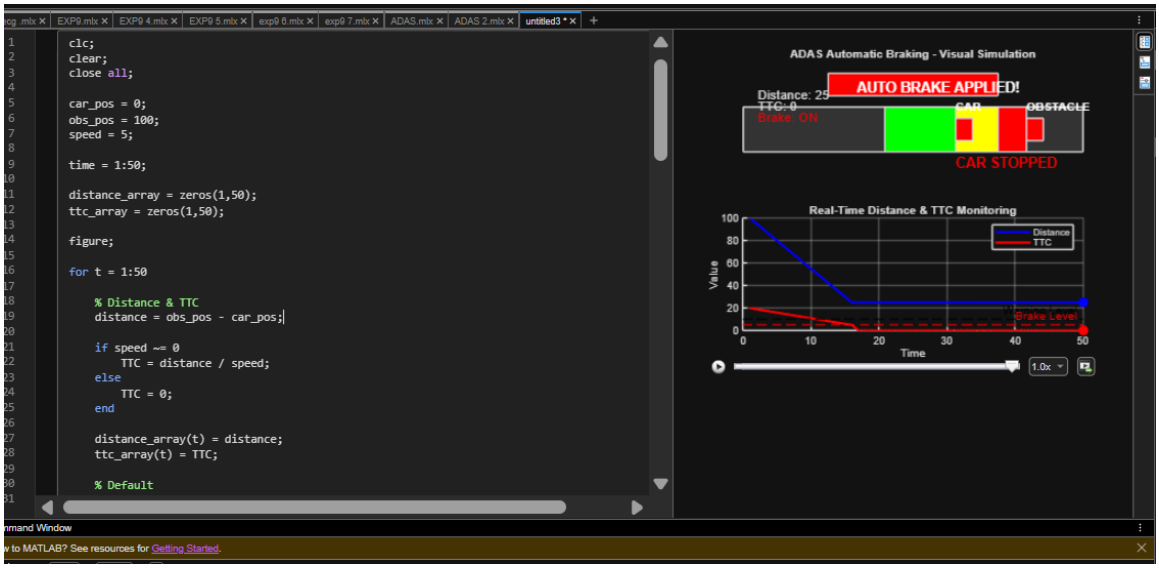
- Sensor Module: A mathematical representation of LiDAR/Radar that provides continuous distance arrays.
- Processing Unit: The main MATLAB script that handles the while-loop and data routing.
- TTC Calculation Module: A dedicated function that processes distance and velocity arrays to output time variables.
- Decision Unit: The logical framework (if/else structures) that acts as the vehicle's "brain."
- Actuator (Braking): The mathematical deceleration applied to the vehicle's speed variable, simulating physical brake calipers clamping.
- Dashboard Display: MATLAB figure windows serving as the graphical user interface (GUI) for the driver.



6. Results & Observations

Running the simulation yields clear kinematic trends:

- Distance vs. Time: A linear decrease in distance is observed until the warning phase, followed by a curved, asymptotic flattening as the automatic brakes engage and the vehicle halts safely before the obstacle coordinate.
- TTC vs. Time: The TTC drops steadily. The simulation visually highlights the exact timestamps where the warning flag is thrown (at exactly 10 seconds) and where braking forces are applied (at exactly 5 seconds).
- Collision Avoidance: The system successfully brings the vehicle's velocity to zero while the distance variable remains greater than zero, proving that the collision was mathematically avoided.



7. Challenges Encountered

- Threshold Selection: Calibrating the TTC thresholds (5s and 10s) required balancing safety with passenger comfort. Overly aggressive thresholds cause abrupt, unrealistic stops, while relaxed thresholds lead to collisions.
- Discrete Time Steps (Δt): Ensuring the MATLAB simulation updates fast enough (e.g., every 0.1 seconds) to capture sudden changes without overwhelming the computer's memory.
- Smooth Visualization: Creating a continuous, flicker-free animation in MATLAB required careful management of plotting functions and commands.

8. Future Scope

The foundational logic built in this project can be expanded into more complex, real-world applications:

- Hardware Integration: Transitioning the MATLAB logic to a microcontroller (like Raspberry Pi or Arduino) and hooking it up to actual ultrasonic sensors or LiDAR.
- Computer Vision: Implementing AI-based object detection using MATLAB's Automated Driving Toolbox to identify pedestrians, lane markings, and varying vehicle types rather than a simple stationary point.
- Driver Monitoring: Integrating a cabin-facing camera to adjust TTC thresholds based on whether the driver is paying attention or looking at their phone.

- Dynamic Obstacles: Upgrading the simulation to account for moving obstacles, lead-vehicle sudden braking, and complex intersection scenarios.

9. Conclusion

This project successfully demonstrates the core principles of an Advanced Driver Assistance System using MATLAB. By mathematically modeling distance, speed, and Time to Collision, the simulation effectively executes timely warnings and automatic emergency braking. This highlights how algorithmic automation can drastically reduce human dependency in critical driving scenarios, serving as a vital step toward fully autonomous, accident-free transportation.